

This document is a quick guide to set up and run the CSV Processor application locally using Minikube, or through other available approaches such as Docker and AWS-based deployments.

1. Prerequisites

Required for All Environments

- Git
- Docker Desktop

Local / Minikube

- Minikube
- kubectl
- Helm 3
- Python 3.10 or later

AWS Deployments

- AWS CLI v2
- kubectl
- Helm 3
- Git Bash (Windows)
- kops (for kops-based deployment)
- Terraform 1.5+ (for EKS deployment)

Verify AWS credentials:

```
aws sts get-caller-identity
```

2. Cloning the Repository

Clone the repository to your local machine and move into the project folder:

```
git clone https://github.com/asif-ahmedb/csv-processor
cd csv-processor
```

3. Setting Up and Running the App

Follow the steps below to build, test, and run the CSV Processor application locally using Docker or Minikube.

3.1 Run the Application Using Docker

```
cd csv-processor-app
docker build -t csv-processor:local .
docker run --rm -p 8081:8081 \
  -e S3_BUCKET="" \
  -e AWS_REGION=us-east-1 \
  csv-processor:local
```

Access the application at: <http://localhost:8081>

3.2 Run Unit Tests

```
cd csv-processor-app/web
pip install -r requirements.txt
pytest tests/ -v
```

3.3 Deploy on Minikube

```
cd csv-processor-k8s-assets/minikube
./deploy.sh
kubectl port-forward -n csv-processor svc/csv-processor 8080:80
```

Access the application at: <http://localhost:8080>

4. AWS Deployment Using kops

4.1 Configuration

```
cd csv-processor-infrastructure/kops
cp settings.env.example settings.env
```

Update the following values:

```
AWS_ACCOUNT_ID=<your_account_id>
AWS_REGION=us-east-1
```

4.2 Create the Kubernetes Cluster

```
chmod +x scripts/*.sh
./scripts/bootstrap.sh
```

4.3 Deploy the Application

```
cd ../../csv-processor-k8s-assets
source ../csv-processor-infrastructure/kops/settings.env

helm upgrade --install csv-processor ./helm/csv-processor \
  -f ./helm/csv-processor/values-kops.yaml \
  --namespace csv-processor --create-namespace \
  --set config.s3Bucket="${PROCESSED_BUCKET}" \
  --set-string aws.accountId="${AWS_ACCOUNT_ID}"
```

4.4 Access the Application

```
kubectl get svc -n csv-processor csv-processor -w
```

Open the external IP displayed.

4.5 Teardown

```
cd csv-processor-infrastructure/kops
./scripts/teardown.sh --yes
```

5. AWS Deployment Using Terraform / EKS

5.1 Provision Infrastructure

```
cd csv-processor-infrastructure/terraform
cp terraform.tfvars.example terraform.tfvars
terraform init
terraform apply
```

5.2 Configure kubectl

```
aws eks update-kubeconfig \
  --name $(terraform output -raw eks_cluster_name) \
  --region $(terraform output -raw aws_region)
```

5.3 Deploy the Application

```
cd ../../csv-processor-k8s-assets

helm upgrade --install csv-processor ./helm/csv-processor \
  -f ./helm/csv-processor/values-eks.yaml \
  --namespace csv-processor --create-namespace \
  --set config.s3Bucket=$(terraform -chdir=../csv-processor-
infrastructure/terraform output -raw s3_bucket_name) \
```

```
--set aws.irsRoleArn=$(terraform -chdir=./csv-processor-  
infrastructure/terraform output -raw app_irs_role_arn)
```

5.4 Access the Application

```
kubectl port-forward -n csv-processor svc/csv-processor 8080:80
```

Open: <http://localhost:8080>

5.5 Teardown

```
cd csv-processor-infrastructure/terraform  
terraform destroy
```

6. CI/CD Workflow

The GitHub Actions workflow automatically triggers on changes to the `csv-processor-app/` directory. The pipeline performs the following steps:

1. Executes unit tests
2. Builds the Docker image
3. Pushes the image to Docker Hub

Required GitHub secrets:

- DOCKERHUB_USERNAME
- DOCKERHUB_TOKEN